

Virtual Machines vs. Containers

Feature	Virtual Machines (VMs)	Containers
Architecture	Hardware-level virtualization using a Hypervisor.*	OS-level virtualization utilizing the host's Linux Kernel.
Operating System	Each VM runs a complete, independent Guest OS.	All containers share the single Host OS kernel.
Boot Time	Slower (typically minutes) as the full OS must boot.	Extremely fast (milliseconds) as they are treated as standard processes.
Resource Overhead	High; requires dedicated CPU and RAM allocations upon creation.	Low; resources are consumed elastically only when needed.
Isolation Mechanism	Strong isolation managed at the hardware virtualization level.	Process-level isolation managed via Linux Namespaces and Cgroups.

* Pure software-based virtualization (translating instructions via binary translation or emulating hardware entirely in software) is nearly obsolete for standard production environments. We can consider that the percentage is meagre or near 0 for the mainstream enterprise usage. In the case of hyperscale public cloud providers – AWS, Azure, GCP – they do not use pure software-based virtualization for standard virtual machines.

In fact, cloud providers have gone a step further than traditional hardware virtualization. They now use hardware-offloaded virtualization (like AWS Nitro that we discussed during the last contact session.). Instead of the host CPU running the hypervisor software, specialized ASIC(Application-Specific Integrated Circuit) microchips and smartNICs (Smart Network Interface Card) handle the virtualization network, storage, and security management. This frees up 100% of the physical server's CPU to run guest workloads at near-native hardware speeds.

A Kubernetes cluster is divided into two primary layers: the control plane and the worker nodes.

- **The Control Plane:** Consider this like a **Brain**. This manages the global state and scheduling decisions of the entire cluster.
 - **kube-apiserver:** The front-end REST interface that validates and processes all internal and external cluster requests.
 - **etcd:** A distributed, fault-tolerant key-value store acting as the ultimate source of truth for all cluster configuration and state data.
 - **kube-scheduler:** Evaluates resource requirements and constraints to assign incoming pods to the most appropriate compute node.
 - **kube-controller-manager:** Continuously monitors the cluster to ensure the actual running state matches the desired declared state.
- **Worker Nodes:** Consider this like the **Muscle**. The physical or virtual machines that physically execute the application workloads.
 - **Pods:** The smallest deployable object in Kubernetes, representing a single instance of an application running one or more tightly coupled containers.
 - **kubelet:** An agent on each node communicating with the control plane to ensure containers are running correctly inside their assigned pods.
 - **kube-proxy:** A network proxy that facilitates communication and traffic routing inside and outside the cluster.
 - **Container Runtime:** The underlying engine (e.g., containerd, CRI-O) responsible for pulling images and executing the containers.

Cloud-Native Design Principles

Cloud-native technologies empower organizations to build resilient, scalable applications in modern environments like public, private, and hybrid clouds. The Cloud Native Computing Foundation (CNCF) defines this approach using the following architectural pillars:

- **Microservices:** Transitioning from large, monolithic blocks to small, interdependent services that can be scaled and updated independently.
- **Containerization:** Packaging microservices and their dependencies together so they run reliably across any infrastructure without hardware or OS lock-in.
- **Declarative APIs:** Utilizing Application Programming Interfaces to programmatically link loosely coupled services and declare the desired state of the infrastructure.
- **Immutable Infrastructure:** Replacing servers or containers entirely rather than updating them manually in-place, ensuring predictable and repeatable deployments.
- **Service Meshes:** Implementing dedicated infrastructure layers to manage secure, observable, and resilient communication between hundreds of microservices.

Cloud-Native is Cloud-agnostic

The cloud is a model, not a vendor: Being "native" to the cloud means that the software treats infrastructure as an abstract, programmable layer.

Portability via open standards: Kubernetes creates a standard platform layer. We can run it on AWS, on premises in a private data centre, or across a hybrid mix.

Avoiding lock-in: Instead of relying on one provider's specific database or serverless tool, cloud-native tech packages apps into portable containers and microservices that run anywhere.